

Rapport individuel, CHER Naïck

Au cours de cette SAE, réalisée en groupe de 4 avec MURAT Mia, GUEDON Anaëlle et FEVE Maïawella, nous devons réaliser une refonte du site internet d'un club de tennis. J'ai dû user de nombreuses compétences durant la réalisation de ce projet telles que:

-AC21.01 | Elaborer et implémenter les spécifications fonctionnelles et non fonctionnelles à partir des exigences, AC22.01 | Choisir des structures de données complexes adaptées au problème, AC25.02 | Formaliser les besoins du client et de l'utilisateur: La modification du MCD/MLD, et même plus

généralement de la base de données, permet de mieux s'adapter aux exigences du client (cf [Image 1](#)).

-AC24.03 | Organiser la restitution de données à travers la programmation et la visualisation, AC24.04 | Manipuler des données hétérogènes : L'intégration de la base de données en Python grâce à l'API SQLAlchemy permet de manipuler les données facilement sur notre site (cf [Image 2](#)). Les pages concernant les championnats utilisent ces données pour générer des tableaux indiquant les résultats des matchs, le classement des participants du club (cf [Image 3](#)) ou bien le palmarès du club durant une saison (cf [Image 4](#)).

De plus, certaines de ces données peuvent être filtrées, pour les championnats et les articles leur années de début/de publication, et pour les tarifs par sport (cf [Image 5](#)).

-AC26.03 | Mobiliser les compétences interpersonnelles pour travailler dans une équipe informatique : L'utilisation de Github permet de structurer les projets, tout en permettant à chacun de travailler sur ses tâches individuellement et de communiquer pour améliorer son code (cf [Image 6](#)).

-AC21.03 | Adopter de bonnes pratiques de conception et de programmation : La réalisation de tests, de docstrings et des annotations sur mes parties permettent de vérifier que le programme fonctionne comme attendu, en plus de faciliter la tâche pour celui qui veut retoucher au code pour l'améliorer (cf [Image 7](#)).

L'utilisation de Pylint permet en plus de vérifier la qualité du code et de corriger les défauts (cf [Image 8](#)).

Images

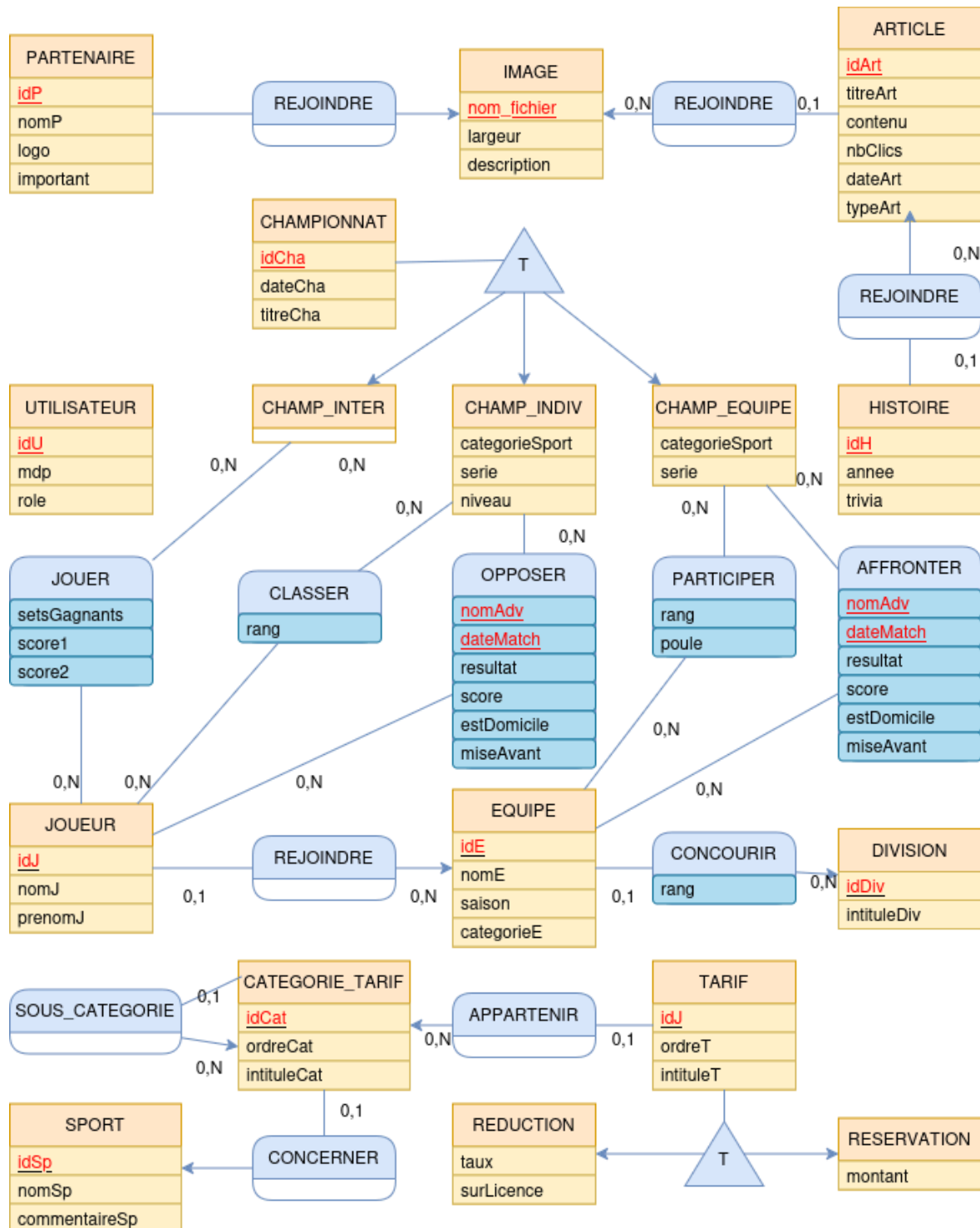


Image 1 : le MCD représentant la base de données sous forme de schéma

```

class Classer(db.Model):
    """Participation/Classement d'un joueur dans un championnat individuel"""
    __tablename__ = "CLASSER"

    id_championnat: int = db.Column("idCha", db.Integer, db.ForeignKey("CHAMP_INDIV.idCha"),
    | primary_key=True)
    id_j: int = db.Column("idJ", db.Integer, db.ForeignKey("JOUEUR.idJ"), primary_key=True)
    rang: str = db.Column("rang", db.Text)

    competition = db.relationship("ChampionnatIndividuel", backref=db.backref("classer",
    | lazy="dynamic", cascade="all, delete-orphan"))
    joueur = db.relationship("Joueur", backref=db.backref("classer",
    | lazy="dynamic", cascade="all, delete-orphan"))

    def __init__(self, id_championnat: int, id_j: int, rang: str):
        self.id_championnat = id_championnat
        self.id_j = id_j
        self.rang = rang

    def __str__(self):
        return f"<Classer({self.id_championnat}, {self.id_j}) {self.rang}>"

    def __repr__(self):
        return self.__str__()

```

Image 2 : le modèle “Classer” permettant de définir les attributs, les clés primaires et étrangères de la table “CLASSER”

Championnat individuel 1

Début le 03/11/2025

- Championnat individuel
- Niveau : Départemental
- Catégorie : Jeune
- Série : Garçons 9 ans Vert

Joueur : Jean Claude			Options
Date	Adversaire	Resultat	(🗑️)(✎)(⊕)
05/10/2025 - Important	Adv1	V 7/0	(🗑️)(✎)
Joueur : Jean Pierre			Options
Date	Adversaire	Resultat	(🗑️)(✎)(⊕)
06/10/2025 - Important	Adv2	V 7/1	(🗑️)(✎)
Joueur : Jean Michel			Options
Date	Adversaire	Resultat	(🗑️)(✎)(⊕)
08/10/2025	Adv3	V 6/2	(🗑️)(✎)

Image 3 : un tableau regroupant les matchs de l’équipe “Stade 1” lors du championnat “Championnat 2023”, indiquant ceux qui sont importants.

Catégorie : Jeune	Vainqueur	Finaliste
Championnat individuel 7	-	-
Championnat individuel 2	-	-
Championnat individuel 1	Jean Claude	Jean Pierre

Catégorie : Séniors +	Vainqueur	Finaliste
Championnat individuel 3	-	-

Catégorie : Séniors	Vainqueur	Finaliste
Championnat individuel 6	-	-

Championnat Individuel Régional

Catégorie : Jeune	Vainqueur	Finaliste
Championnat individuel 4	-	-
Championnat individuel 5	-	-

Catégorie : Séniors +	Vainqueur	Finaliste
Championnat individuel 8	-	-

Championnat par équipe

Equipes jeunes			
championnat equipe 1	Catégorie	Poule	Division
equipe 1	15/18 ans F	1 de la Poule A	1 DSR
championnat equipe 4	Catégorie	Poule	Division
equipe 4	15/18 ans G	4 de la Poule D	4 Division 4

Image 4 : un tableau représentant le palmarès du club lors de l'année 2025, généré automatiquement à partir des données de la base de données

Liste des compétitions de l'année 2023

2023

Compétitions individuelles

Ajouter un championnat individuel

Compétitions par équipes

- 

26/03/2023 - [Championnat 2023](#)

Ajouter un championnat par équipe

Image 5 : La liste des compétitions, filtrée de sorte à n'afficher que les compétitions ayant commencées en 2023.

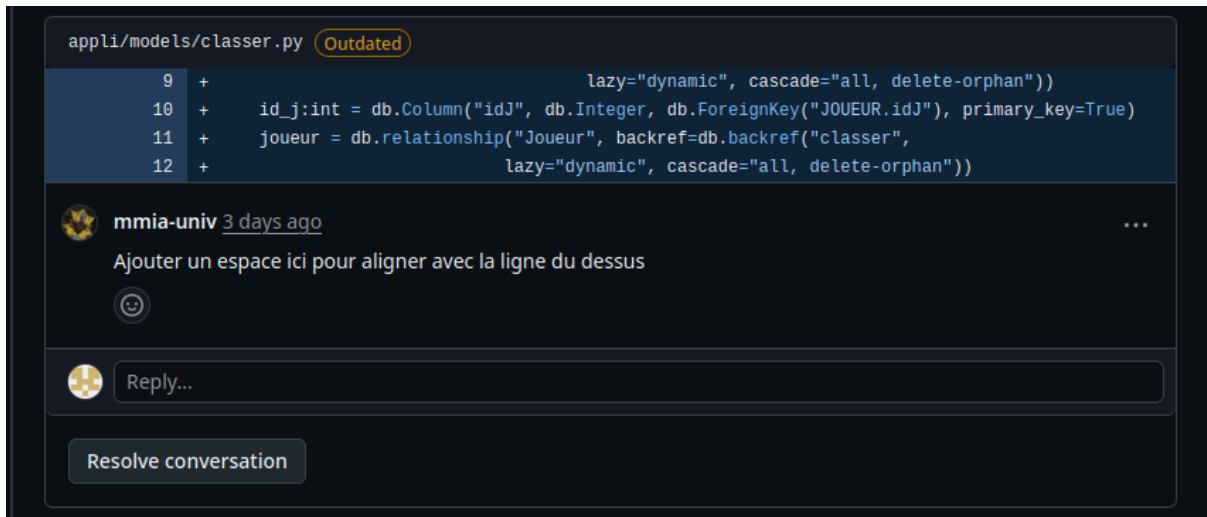
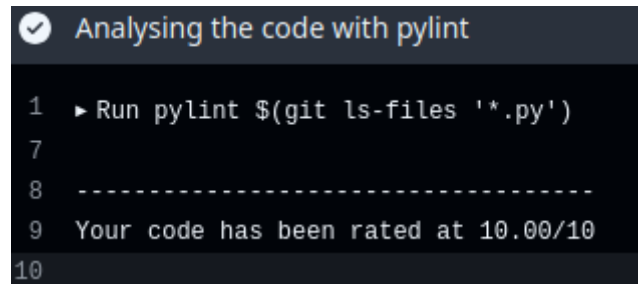


Image 6 : Réponse à un pull-request sur Github, le “reviewer” (un autre membre du groupe), peut donner son avis sur le code envoyé et d’indiquer les modifications nécessaires avant de le valider

Name	Stmts	Miss	Cover	Missing
-----	-----	-----	-----	-----
appli/app.py	14	1	93%	18
appli/commands.py	271	12	96%	415-420, 455-460
appli/forms.py	140	13	91%	93-96, 105-107, 162-164, 195-197
appli/models/affronter.py	26	3	88%	38-40, 43
appli/models/article.py	22	0	100%	
appli/models/categorie_tarif.py	16	0	100%	
appli/models/championnat.py	61	0	100%	
appli/models/classer.py	16	0	100%	
appli/models/division.py	11	0	100%	
appli/models/equipe.py	21	0	100%	
appli/models/histoire.py	15	0	100%	
appli/models/joueur.py	16	0	100%	
appli/models/partenaire.py	15	0	100%	
appli/models/participer.py	18	0	100%	
appli/models/reduction.py	15	2	87%	22, 25
appli/models/reservation.py	13	2	85%	20, 23
appli/models/tarif.py	14	0	100%	
appli/models/utilisateur.py	13	0	100%	
appli/statique.py	19	0	100%	
appli/views/articles.py	52	9	83%	30-35, 52-54, 69
appli/views/autre.py	5	0	100%	
appli/views/competitions.py	295	1	99%	504
appli/views/contacts.py	93	16	83%	17-19, 108-121
appli/views/documents.py	5	0	100%	
appli/views/ecole.py	5	0	100%	
appli/views/error.py	11	1	91%	15
appli/views/histoire.py	51	4	92%	48-52
appli/views/index.py	8	0	100%	
appli/views/logout.py	7	2	71%	10-11
appli/views/management.py	5	0	100%	
appli/views/partenaires.py	33	13	61%	25-29, 38-45
appli/views/redirect.py	11	0	100%	
appli/views/tarifications.py	106	0	100%	
appli/views/users.py	59	0	100%	
-----	-----	-----	-----	-----
TOTAL	1482	79	95%	

Image 7 : Le rapport de test, indiquant la couverture des tests qui est ici de 95%

A terminal window with a dark background and light-colored text. The title bar at the top says "Analysing the code with pylint" with a checkmark icon. The terminal content shows a command being run, followed by a separator line of dashes, and a final message indicating a perfect score of 10.00/10.

```
1  ► Run pylint $(git ls-files '*.py')
7
8  -----
9  Your code has been rated at 10.00/10
10
```

Image 8 : Analyse du code par pylint, indiquant si il y a des défauts au niveau de la qualité du code, une note de 10 indique que le code est de bonne qualité